Héctor Pons Castellano, Raúl Martínez Vilanova, Zeus Mallenco Tolosa

EJERCICIO 1:

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <sstream>
#include <unistd.h> //Necesario para execvp

using namespace std;

int main() {
   //Imprimimos el prompt "? " y forzamos su salida a pantalla
   cout<<"? "<<flush;

   //Leemos la línea completa que escribe el usuario
   string linea;
   if (!getline(cin, linea))
       return 0; //si hay un error de lectura salimos


   //Troceamos la linea para separar el comando de sus argumentos
   istringstream flujo(linea);
   string palabra;
   vector<string> argumentos_str;

   //El flujo extrae palabra por palabra ignorando los espacios automaticamente
   while (flujo >> palabra)
       argumentos_str.push_back(palabra);


   //Comprobamos si la línea estaba vacía (solo pulsó INTRO)
   if (argumentos_str.empty())
       return 0;

   //Preparamos el array tipo C que exige execvp
   vector<char*> args;
   for (size_t i = 0; i < argumentos_str.size(); i++)
   {
       //Accedemos a cada palabra usando el índice [i]
       args.push_back(const_cast<char*>(argumentos_str[i].c_str()));
   }
   //El ultimo elemento del array para execvp debe ser siempre nulo
   args.push_back(nullptr);

   //Llamada al sistema execvp
   //args[0]: nombre del comando (ej: "date")
   //args.data(): entrega el array subyacente compatible con C
```

```
    execvp(args[0], args.data());

    //Si el código llega aquí, execvp falla (por ejempplo por un comando inventado)
    perror("Error");
    return 1;
}
```

```
user1@base:~$ g++ ej1.cpp -o ej1
user1@base:~$ ./ej1.cpp
bash: ./ej1.cpp: Permission denied
user1@base:~$ ./ej1
? date
Thu Mar 12 12:46:37 PM CET 2026
user1@base:~$ echo uno dos 3
uno dos 3
user1@base:~$
```

EJERCICIO 2:

```
#include <iostream>
#include <string>
#include <vector>
#include <sstream>
#include <unistd.h> //Necesario para execvp
#include <sys/wait.h>

using namespace std;
int main() {
    //Imprimimos el prompt "? " y forzamos su salida a pantalla
    cout<<"? "<<flush;

    //Leemos la línea completa que escribe el usuario
    string linea;
    if (!getline(cin, linea))
        return 0; //si hay un error de lectura salimos


    //Troceamos la línea para separar el comando de sus argumentos
    istringstream flujo(linea);
    string palabra;
    vector<string> argumentos_str;

    //El flujo extrae palabra por palabra ignorando los espacios automaticamente
    while (flujo >> palabra)
        argumentos_str.push_back(palabra);
```

```
    //Comprobamos si la línea estaba vacía (solo pulsó INTRO)
    if (argumentos_str.empty())
        return 0;

    //Preparamos el array tipo C que exige execvp
    vector<char*> args;
           for (size_t i = 0; i < argumentos_str.size(); i++)
    {
        //Accedemos a cada palabra usando el índice [i]
        args.push_back(const_cast<char*>(argumentos_str[i].c_str()));
    }
    //El ultimo elemento del array para execvp debe ser siempre nulo
    args.push_back(nullptr);

    cout<<"padre: creando copia"<< endl;
    pid_t pid = fork();

    if (pid < 0) {
        //Si pid es menor que 0 significa que ha flallado al clonar
        perror("Error al hacer fork");
        return 1;
    }
    else if(pid==0){
        cout<< "hijo: a punto de hacer exec"<<endl;
        execvp(args[0], args.data()); //sacrificamos al hijo

        //Si llegamos aqui es que el exec ha fallado
        perror("execvp");
        return 1;
    }else{
        cout<<"padre: esperando al hijo"<<endl;
        //Esperamos a que el hijo termine
        wait(NULL);
        cout<<"padre: el hijo ha terminado, finalizando"<< endl;
    }

    return 0;
}
```

```
user1@base:~$ g++ ej2.cpp -o ej2
user1@base:~$ ./ej2
? hola
padre: creando copia
padre: esperando al hijo
hijo: a punto de hacer exec
execvp: No such file or directory
padre: el hijo ha terminado, finalizando
user1@base:~$ ./ej2
? date
padre: creando copia
padre: esperando al hijo
hijo: a punto de hacer exec
Thu Mar 12 01:18:17 PM CET 2026
padre: el hijo ha terminado, finalizando
user1@base:~$
```

EJERCICIO 3:

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <sstream>
#include <unistd.h> //Necesario para execvp
#include <sys/wait.h>

using namespace std;

int main() {
   string linea;

   while(true)
   {
   //Imprimimos el prompt "? " y forzamos su salida a pantalla
   cout<<"? "<<flush;

   //Leemos la línea completa que escribe el usuario,
   if (!getline(cin, linea))
   {
       cout<<endl;
       break;
   }

   //Troceamos la linea para separar el comando de sus argumentos
   istringstream flujo(linea);
   string palabra;
   vector<string> argumentos_str;

   //El flujo extrae palabra por palabra ignorando los espacios automaticamente
```

```
    while (flujo >> palabra)
        argumentos_str.push_back(palabra);

    // Comprobamos si la línea estaba vacía (intro)
    if (argumentos_str.empty())
        continue;

    //Preparamos el array tipo C que exige execvp
    vector<char*> args;
    for (size_t i = 0; i < argumentos_str.size(); i++)
    {
        //Accedemos a cada palabra usando el índice [i]
        args.push_back(const_cast<char*>(argumentos_str[i].c_str()));
    }
    //El ultimo elemento del array para execvp debe ser siempre nulo
    args.push_back(nullptr);

    cout<<"padre: creando copia"<< endl;
    pid_t pid = fork();

    if (pid < 0) {
        //Si pid es menor que 0 significa que ha flallado al clonar
        perror("Error al hacer fork");
        return 1;
    }
    else if(pid==0){
        cout<< "hijo: a punto de hacer exec"<<endl;
        execvp(args[0], args.data()); //sacrificamos al hijo

        //si llegamos aqui es que el exec ha fallado
        perror("execvp");
        return 1;
    }else{
        cout<<"padre: esperando al hijo"<<endl;
        //Esperamos a que el hijo termine
        wait(NULL);
        cout<<"padre: el hijo ha terminado, finalizando"<< endl;
    }

    }

    return 0;
}
```

```
user1@base:~$ g++ ej3.cpp -o ej3
user1@base:~$ ./ej3
? hola
padre: creando copia
padre: esperando al hijo
hijo: a punto de hacer exec
execvp: No such file or directory
padre: el hijo ha terminado, finalizando
? date
padre: creando copia
padre: esperando al hijo
hijo: a punto de hacer exec
Thu Mar 12 01:36:44 PM CET 2026
padre: el hijo ha terminado, finalizando
?
user1@base:~$
```

EJERCICIO 4:

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <sstream>
#include <unistd.h> //Necesario para execvp
#include <sys/wait.h>

using namespace std;

int main() {
   string linea;

   while(true)
   {

   //mmprimimos el prompt "? " y forzamos su salida a pantalla
   cout<<"? "<<flush;

   //leemos la línea completa que escribe el usuario,
   if (!getline(cin, linea))
   {
       cout<<endl;
       break;
   }


   //Troceamos la linea para separar el comando de sus argumentos
   istringstream flujo(linea);
   string palabra;
   vector<string> argumentos_str;
```

```
// El flujo extrae palabra por palabra ignorando los espacios automaticamente
while (flujo >> palabra)
    argumentos_str.push_back(palabra);


// 4. Comprobamos si la línea estaba vacía (solo pulsó INTRO)
if (argumentos_str.empty())
    continue;

string comando = argumentos_str[0];

if (comando == "exit")
{
        break; // termina el programa
}else if (comando == "cd")
{
        if (argumentos_str.size() > 1)
        {
            // chdir() pide un const char*, así que usamos .c_str()
            int resultado = chdir(argumentos_str[1].c_str());
            if (resultado!=0)
            {
                perror("cd"); //Si la carpeta no existe, avisa
            }
        } else
        {
            cout << "cd: especifica un directorio" << endl;
        }
        continue;
}else if (comando == "exec")
{
        if (argumentos_str.size() > 1)
        {
            vector<char*> args;
            for (size_t i = 1; i < argumentos_str.size(); i++) {
                args.push_back(const_cast<char*>(argumentos_str[i].c_str()));
            }
            args.push_back(nullptr);

            execvp(args[0], args.data());

            perror("execvp");
        } else
        {
            cout << "exec: especifica un comando" << endl;
```

```
            }
            continue; // Si fallaj volvemos a pedir orden
        }


    //Preparamos el array tipo C que exige execvp
    vector<char*> args;
    for (size_t i = 0; i < argumentos_str.size(); i++)
    {
        //Accedemos a cada palabra usando el índice [i]
        args.push_back(const_cast<char*>(argumentos_str[i].c_str()));
    }
    //el ultimo elemento del array para execvp debe ser siempre nulo
    args.push_back(nullptr);

    cout<<"padre: creando copia"<< endl;
    pid_t pid = fork();

    if (pid < 0) {
        // Si pid es menor que 0 significa que ha flallado al clonar
        perror("Error al hacer fork");
        return 1;
    }
    else if(pid==0){
        cout<< "hijo: a punto de hacer exec"<<endl;
        execvp(args[0], args.data()); //sacrificamos al hijo

        //si llegamos aqui es que el exec ga fallado
        perror("execvp");
        return 1;
    }else{
        cout<<"padre: esperando al hijo"<<endl;
        //Esperamos a que el hijo termine
        wait(NULL);
        cout<<"padre: el hijo ha terminado, finalizando"<< endl;
    }

    }

    return 0;
}
```